

Pandas Lab Assignment with Solutions

Instructions: Use **import pandas as pd** in all questions. Each question includes its own data and solution.

Section A - DataFrame Basics (Q1-Q10)

Q1. Create a DataFrame from the given dictionary and display it.

Data

```
data = {'Name': ['Aman', 'Riya', 'Karan'], 'Marks': [78, 85, 90]}
```

Solution

```
import pandas as pd
data = {'Name': ['Aman', 'Riya', 'Karan'], 'Marks': [78, 85, 90]}
df = pd.DataFrame(data)
print(df)
```

Q2. Create a DataFrame from the given list of dictionaries.

Data

```
data = [
    {'Name': 'Aman', 'Age': 20},
    {'Name': 'Riya', 'Age': 21},
    {'Name': 'Karan', 'Age': 19}
]
```

Solution

```
import pandas as pd
data = [
    {'Name': 'Aman', 'Age': 20},
    {'Name': 'Riya', 'Age': 21},
    {'Name': 'Karan', 'Age': 19}
]
df = pd.DataFrame(data)
print(df)
```

Q3. Create a DataFrame and display the first 2 rows using head().

Data

```
data = {'City': ['Delhi', 'Patiala', 'Ludhiana', 'Amritsar'], 'Temp': [35, 33, 30, 28]}
```

Solution

```
import pandas as pd
data = {'City': ['Delhi', 'Patiala', 'Ludhiana', 'Amritsar'], 'Temp': [35, 33, 30, 28]}
df = pd.DataFrame(data)
print(df.head(2))
```

Q4. Create a DataFrame and display the last 2 rows using tail().

Data

```
data = {'Product': ['Pen', 'Pencil', 'Eraser', 'Scale'], 'Price': [10, 5, 8, 15]}
```

Solution

```
import pandas as pd
data = {'Product': ['Pen', 'Pencil', 'Eraser', 'Scale'], 'Price': [10, 5, 8, 15]}
df = pd.DataFrame(data)
print(df.tail(2))
```

Q5. Create a DataFrame and display the column names.

Data

```
data = {'RollNo': [1, 2, 3], 'Name': ['A', 'B', 'C'], 'CGPA': [8.2, 8.5, 9.0]}
```

Solution

```
import pandas as pd
data = {'RollNo': [1, 2, 3], 'Name': ['A', 'B', 'C'], 'CGPA': [8.2, 8.5, 9.0]}
df = pd.DataFrame(data)
print(df.columns)
```

Q6. Create a DataFrame and display the index values.

Data

```
data = {'Name': ['Rohit', 'Mohit', 'Sumit'], 'Score': [45, 56, 67]}
```

Solution

```
import pandas as pd
data = {'Name': ['Rohit', 'Mohit', 'Sumit'], 'Score': [45, 56, 67]}
df = pd.DataFrame(data)
print(df.index)
```

Q7. Create a DataFrame and display data types of all columns.

Data

```
data = {'Name': ['Aman', 'Riya'], 'Age': [20, 21], 'Fee': [25000.5, 27000.0]}
```

Solution

```
import pandas as pd
data = {'Name': ['Aman', 'Riya'], 'Age': [20, 21], 'Fee': [25000.5, 27000.0]}
df = pd.DataFrame(data)
print(df.dtypes)
```

Q8. Create a DataFrame and display its shape.

Data

```
data = {'A': [1, 2, 3], 'B': [4, 5, 6], 'C': [7, 8, 9]}
```

Solution

```
import pandas as pd
data = {'A': [1, 2, 3], 'B': [4, 5, 6], 'C': [7, 8, 9]}
df = pd.DataFrame(data)
print(df.shape)
```

Q9. Create a DataFrame and display all values as NumPy representation.

Data

```
data = {'Item': ['Book', 'Bag'], 'Cost': [250, 500]}
```

Solution

```
import pandas as pd
data = {'Item': ['Book', 'Bag'], 'Cost': [250, 500]}
df = pd.DataFrame(data)
print(df.values)
```

Q10. Create a DataFrame and display concise summary using info().

Data

```
data = {'Name': ['A', 'B', 'C'], 'Marks': [67, 89, 76], 'Passed': [True, True, True]}
```

Solution

```
import pandas as pd
data = {'Name': ['A', 'B', 'C'], 'Marks': [67, 89, 76], 'Passed': [True, True, True]}
df = pd.DataFrame(data)
df.info()
```

Section B - Adding, Deleting, Renaming, Missing Values (Q11-Q20)

Q11. Add a new column Total to the DataFrame.

Data

```
data = {'Name': ['Aman', 'Riya'], 'Math': [80, 90], 'Science': [85, 88]}
```

Solution

```
import pandas as pd
data = {'Name': ['Aman', 'Riya'], 'Math': [80, 90], 'Science': [85, 88]}
df = pd.DataFrame(data)
df['Total'] = df['Math'] + df['Science']
print(df)
```

Q12. Add a new row to the DataFrame using loc.

Data

```
data = {'Name': ['Aman', 'Riya'], 'Age': [20, 21]}
```

Solution

```
import pandas as pd
data = {'Name': ['Aman', 'Riya'], 'Age': [20, 21]}
df = pd.DataFrame(data)
df.loc[2] = ['Karan', 22]
print(df)
```

Q13. Delete the column Age.

Data

```
data = {'Name': ['Aman', 'Riya'], 'Age': [20, 21], 'City': ['Patiala', 'Delhi']}
```

Solution

```
import pandas as pd
data = {'Name': ['Aman', 'Riya'], 'Age': [20, 21], 'City': ['Patiala', 'Delhi']}
df = pd.DataFrame(data)
df = df.drop('Age', axis=1)
print(df)
```

Q14. Delete the row with index 1.

Data

```
data = {'Name': ['Aman', 'Riya', 'Karan'], 'Marks': [70, 80, 90]}
```

Solution

```
import pandas as pd
data = {'Name': ['Aman', 'Riya', 'Karan'], 'Marks': [70, 80, 90]}
df = pd.DataFrame(data)
df = df.drop(1)
print(df)
```

Q15. Rename column Marks to Score.

Data

```
data = {'Name': ['Aman', 'Riya'], 'Marks': [78, 88]}
```

Solution

```
import pandas as pd
data = {'Name': ['Aman', 'Riya'], 'Marks': [78, 88]}
df = pd.DataFrame(data)
df = df.rename(columns={'Marks': 'Score'})
print(df)
```

Q16. Use append()-style task to add a row. Use concat() for newer pandas versions.

Data

```
data = {'Name': ['Aman', 'Riya'], 'Marks': [78, 88]}
new_row = {'Name': 'Karan', 'Marks': 91}
```

Solution

```
import pandas as pd
data = {'Name': ['Aman', 'Riya'], 'Marks': [78, 88]}
df = pd.DataFrame(data)
new_row = pd.DataFrame([{'Name': 'Karan', 'Marks': 91}])
df = pd.concat([df, new_row], ignore_index=True)
print(df)
```

Q17. Check missing values using isnull().

Data

```
data = {'Name': ['Aman', None, 'Karan'], 'Marks': [78, 88, None]}
```

Solution

```
import pandas as pd
data = {'Name': ['Aman', None, 'Karan'], 'Marks': [78, 88, None]}
df = pd.DataFrame(data)
print(df.isnull())
```

Q18. Drop rows containing missing values using dropna().

Data

```
data = {'Name': ['Aman', None, 'Karan'], 'Marks': [78, 88, None]}
```

Solution

```
import pandas as pd
data = {'Name': ['Aman', None, 'Karan'], 'Marks': [78, 88, None]}
df = pd.DataFrame(data)
print(df.dropna())
```

Q19. Fill missing values with 0 using fillna().

Data

```
data = {'A': [1, None, 3], 'B': [None, 5, 6]}
```

Solution

```
import pandas as pd
data = {'A': [1, None, 3], 'B': [None, 5, 6]}
df = pd.DataFrame(data)
print(df.fillna(0))
```

Q20. Merge two DataFrames on column ID.

Data

```
df1 = pd.DataFrame({'ID': [1, 2, 3], 'Name': ['Aman', 'Riya', 'Karan']})
df2 = pd.DataFrame({'ID': [1, 2, 3], 'Marks': [78, 88, 91]})
```

Solution

```
import pandas as pd
df1 = pd.DataFrame({'ID': [1, 2, 3], 'Name': ['Aman', 'Riya', 'Karan']})
df2 = pd.DataFrame({'ID': [1, 2, 3], 'Marks': [78, 88, 91]})
df = pd.merge(df1, df2, on='ID')
print(df)
```

Section C - File Handling (Q21-Q25)

Q21. Create a CSV file from given data and read it using read_csv().

Data

```
data = {'Name': ['Aman', 'Riya'], 'Age': [20, 21]}
```

Solution

```
import pandas as pd
data = {'Name': ['Aman', 'Riya'], 'Age': [20, 21]}
df = pd.DataFrame(data)
df.to_csv('student.csv', index=False)

df2 = pd.read_csv('student.csv')
print(df2)
```

Q22. Create a JSON file from given data and read it using read_json().

Data

```
data = {'Name': ['Aman', 'Riya'], 'Marks': [78, 88]}
```

Solution

```
import pandas as pd
data = {'Name': ['Aman', 'Riya'], 'Marks': [78, 88]}
df = pd.DataFrame(data)
df.to_json('student.json')

df2 = pd.read_json('student.json')
print(df2)
```

Q23. Read a CSV file and display the DataFrame.

Data

```
data = {'City': ['Patiala', 'Delhi'], 'Population': [500000, 19000000]}
```

Solution

```
import pandas as pd
data = {'City': ['Patiala', 'Delhi'], 'Population': [500000, 19000000]}
df = pd.DataFrame(data)
df.to_csv('city.csv', index=False)

df2 = pd.read_csv('city.csv')
print(df2)
```

Q24. Read a CSV file and display first 1 row.

Data

```
data = {'Item': ['Pen', 'Pencil', 'Scale'], 'Price': [10, 5, 15]}
```

Solution

```
import pandas as pd
data = {'Item': ['Pen', 'Pencil', 'Scale'], 'Price': [10, 5, 15]}
df = pd.DataFrame(data)
df.to_csv('items.csv', index=False)

df2 = pd.read_csv('items.csv')
print(df2.head(1))
```

Q25. Read a CSV file and display info().

Data

```
data = {'Name': ['A', 'B', 'C'], 'Age': [20, 21, 22]}
```

Solution

```
import pandas as pd
data = {'Name': ['A', 'B', 'C'], 'Age': [20, 21, 22]}
df = pd.DataFrame(data)
df.to_csv('infofile.csv', index=False)

df2 = pd.read_csv('infofile.csv')
df2.info()
```

Section D - Mathematical and Statistical Functions (Q26-Q35)

Q26. Add two DataFrames using add().

Data

```
df1 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
df2 = pd.DataFrame({'A': [5, 6], 'B': [7, 8]})
```

Solution

```
import pandas as pd
df1 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
df2 = pd.DataFrame({'A': [5, 6], 'B': [7, 8]})
print(df1.add(df2))
```


Q27. Subtract one DataFrame from another using sub().

Data

```
df1 = pd.DataFrame({'A': [10, 20], 'B': [30, 40]})  
df2 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})
```

Solution

```
import pandas as pd  
df1 = pd.DataFrame({'A': [10, 20], 'B': [30, 40]})  
df2 = pd.DataFrame({'A': [1, 2], 'B': [3, 4]})  
print(df1.sub(df2))
```

Q28. Multiply two DataFrames using mul().

Data

```
df1 = pd.DataFrame({'A': [2, 3], 'B': [4, 5]})  
df2 = pd.DataFrame({'A': [10, 20], 'B': [2, 3]})
```

Solution

```
import pandas as pd  
df1 = pd.DataFrame({'A': [2, 3], 'B': [4, 5]})  
df2 = pd.DataFrame({'A': [10, 20], 'B': [2, 3]})  
print(df1.mul(df2))
```

Q29. Divide one DataFrame by another using div().

Data

```
df1 = pd.DataFrame({'A': [20, 30], 'B': [40, 50]})  
df2 = pd.DataFrame({'A': [2, 3], 'B': [4, 5]})
```

Solution

```
import pandas as pd  
df1 = pd.DataFrame({'A': [20, 30], 'B': [40, 50]})  
df2 = pd.DataFrame({'A': [2, 3], 'B': [4, 5]})  
print(df1.div(df2))
```

Q30. Find sum of all numeric columns.

Data

```
data = {'A': [1, 2, 3], 'B': [4, 5, 6]}
```

Solution

```
import pandas as pd  
data = {'A': [1, 2, 3], 'B': [4, 5, 6]}  
df = pd.DataFrame(data)  
print(df.sum())
```

Q31. Use agg() to find sum of numeric columns.

Data

```
data = {'A': [2, 4, 6], 'B': [1, 3, 5]}
```

Solution

```
import pandas as pd
data = {'A': [2, 4, 6], 'B': [1, 3, 5]}
df = pd.DataFrame(data)
print(df.agg('sum'))
```

Q32. Find percentage change using pct_change().

Data

```
data = {'Sales': [100, 120, 150, 180]}
```

Solution

```
import pandas as pd
data = {'Sales': [100, 120, 150, 180]}
df = pd.DataFrame(data)
print(df.pct_change())
```

Q33. Find covariance between two series.

Data

```
s1 = pd.Series([10, 20, 30, 40])
s2 = pd.Series([12, 24, 33, 45])
```

Solution

```
import pandas as pd
s1 = pd.Series([10, 20, 30, 40])
s2 = pd.Series([12, 24, 33, 45])
print(s1.cov(s2))
```

Q34. Find correlation between two series.

Data

```
s1 = pd.Series([1, 2, 3, 4, 5])
s2 = pd.Series([2, 4, 6, 8, 10])
```

Solution

```
import pandas as pd
s1 = pd.Series([1, 2, 3, 4, 5])
s2 = pd.Series([2, 4, 6, 8, 10])
print(s1.corr(s2))
```

Q35. Display descriptive statistics using describe().

Data

```
data = {'Math': [78, 88, 92, 85], 'Science': [81, 79, 95, 89]}
```

Solution

```
import pandas as pd
data = {'Math': [78, 88, 92, 85], 'Science': [81, 79, 95, 89]}
df = pd.DataFrame(data)
print(df.describe())
```

Section E - Sorting (Q36-Q45)

Q36. Sort DataFrame by column Marks in ascending order.

Data

```
data = {'Name': ['Aman', 'Riya', 'Karan'], 'Marks': [78, 95, 85]}
```

Solution

```
import pandas as pd
data = {'Name': ['Aman', 'Riya', 'Karan'], 'Marks': [78, 95, 85]}
df = pd.DataFrame(data)
print(df.sort_values('Marks'))
```

Q37. Sort DataFrame by column Marks in descending order.

Data

```
data = {'Name': ['Aman', 'Riya', 'Karan'], 'Marks': [78, 95, 85]}
```

Solution

```
import pandas as pd
data = {'Name': ['Aman', 'Riya', 'Karan'], 'Marks': [78, 95, 85]}
df = pd.DataFrame(data)
print(df.sort_values('Marks', ascending=False))
```

Q38. Sort by multiple columns Class and Marks.

Data

```
data = {
    'Name': ['A', 'B', 'C', 'D'],
    'Class': ['X', 'X', 'Y', 'Y'],
    'Marks': [75, 88, 75, 90]
}
```

Solution

```
import pandas as pd
data = {
    'Name': ['A', 'B', 'C', 'D'],
    'Class': ['X', 'X', 'Y', 'Y'],
    'Marks': [75, 88, 75, 90]
}
df = pd.DataFrame(data)
print(df.sort_values(['Class', 'Marks']))
```

Q39. Sort DataFrame by index.

Data

```
data = {'Name': ['A', 'B', 'C'], 'Marks': [70, 80, 90]}
```

Solution

```
import pandas as pd
data = {'Name': ['A', 'B', 'C'], 'Marks': [70, 80, 90]}
df = pd.DataFrame(data, index=[2, 0, 1])
print(df.sort_index())
```

Q40. Sort by multiple columns with different sort orders.

Data

```
data = {
    'Name': ['A', 'B', 'C', 'D'],
    'Section': ['A', 'A', 'B', 'B'],
    'Marks': [80, 70, 90, 85]
}
```

Solution

```
import pandas as pd
data = {
    'Name': ['A', 'B', 'C', 'D'],
    'Section': ['A', 'A', 'B', 'B'],
    'Marks': [80, 70, 90, 85]
}
df = pd.DataFrame(data)
print(df.sort_values(['Section', 'Marks'], ascending=[True, False]))
```

Q41. Sort values and ignore old index.

Data

```
data = {'Name': ['Aman', 'Riya', 'Karan'], 'Marks': [50, 95, 75]}
```

Solution

```
import pandas as pd
data = {'Name': ['Aman', 'Riya', 'Karan'], 'Marks': [50, 95, 75]}
df = pd.DataFrame(data)
print(df.sort_values('Marks', ignore_index=True))
```

Q42. Sort values using mergesort.

Data

```
data = {'A': [3, 1, 2], 'B': [9, 8, 7]}
```

Solution

```
import pandas as pd
data = {'A': [3, 1, 2], 'B': [9, 8, 7]}
df = pd.DataFrame(data)
print(df.sort_values('A', kind='mergesort'))
```

Q43. Sort values keeping null values first.

Data

```
data = {'Marks': [78, None, 90, 65]}
```

Solution

```
import pandas as pd
data = {'Marks': [78, None, 90, 65]}
df = pd.DataFrame(data)
print(df.sort_values('Marks', na_position='first'))
```

Q44. Sort values keeping null values last.

Data

```
data = {'Marks': [78, None, 90, 65]}
```

Solution

```
import pandas as pd
data = {'Marks': [78, None, 90, 65]}
df = pd.DataFrame(data)
print(df.sort_values('Marks', na_position='last'))
```

Q45. Sort DataFrame by column Price.

Data

```
data = {'Item': ['Pen', 'Book', 'Bag'], 'Price': [10, 250, 500]}
```

Solution

```
import pandas as pd
data = {'Item': ['Pen', 'Book', 'Bag'], 'Price': [10, 250, 500]}
df = pd.DataFrame(data)
print(df.sort_values('Price'))
```

Section F - Data Extraction (Q46-Q55)

Q46. Extract single column Name.

Data

```
data = {'Name': ['Aman', 'Riya', 'Karan'], 'Marks': [78, 88, 91]}
```

Solution

```
import pandas as pd
data = {'Name': ['Aman', 'Riya', 'Karan'], 'Marks': [78, 88, 91]}
df = pd.DataFrame(data)
print(df['Name'])
```

Q47. Extract multiple columns Name and Marks.

Data

```
data = {'Name': ['Aman', 'Riya', 'Karan'], 'Marks': [78, 88, 91], 'Age': [20, 21, 22]}
```

Solution

```
import pandas as pd
data = {'Name': ['Aman', 'Riya', 'Karan'], 'Marks': [78, 88, 91], 'Age': [20, 21, 22]}
df = pd.DataFrame(data)
print(df[['Name', 'Marks']])
```

Q48. Extract row with label 1 using loc.

Data

```
data = {'Name': ['Aman', 'Riya', 'Karan'], 'Marks': [78, 88, 91]}
```

Solution

```
import pandas as pd
data = {'Name': ['Aman', 'Riya', 'Karan'], 'Marks': [78, 88, 91]}
df = pd.DataFrame(data)
print(df.loc[1])
```

Q49. Extract row at position 2 using iloc.

Data

```
data = {'Name': ['Aman', 'Riya', 'Karan'], 'Marks': [78, 88, 91]}
```

Solution

```
import pandas as pd
data = {'Name': ['Aman', 'Riya', 'Karan'], 'Marks': [78, 88, 91]}
df = pd.DataFrame(data)
print(df.iloc[2])
```

Q50. Extract rows from position 1 to 2 using iloc.

Data

```
data = {'Name': ['A', 'B', 'C', 'D'], 'Marks': [60, 70, 80, 90]}
```

Solution

```
import pandas as pd
data = {'Name': ['A', 'B', 'C', 'D'], 'Marks': [60, 70, 80, 90]}
df = pd.DataFrame(data)
print(df.iloc[1:3])
```

Q51. Extract rows 1 to 2 and columns 0 to 1 using iloc.

Data

```
data = {'Name': ['A', 'B', 'C', 'D'], 'Marks': [60, 70, 80, 90], 'Age': [18, 19, 20, 21]}
```

Solution

```
import pandas as pd
data = {'Name': ['A', 'B', 'C', 'D'], 'Marks': [60, 70, 80, 90], 'Age': [18, 19, 20, 21]}
df = pd.DataFrame(data)
print(df.iloc[1:3, 0:2])
```

Q52. Extract rows where Marks > 80.

Data

```
data = {'Name': ['Aman', 'Riya', 'Karan'], 'Marks': [78, 88, 91]}
```

Solution

```
import pandas as pd
data = {'Name': ['Aman', 'Riya', 'Karan'], 'Marks': [78, 88, 91]}
df = pd.DataFrame(data)
print(df[df['Marks'] > 80])
```

Q53. Extract rows where Marks > 70 and Age < 22.

Data

```
data = {'Name': ['Aman', 'Riya', 'Karan'], 'Marks': [78, 88, 91], 'Age': [22, 21, 20]}
```

Solution

```
import pandas as pd
data = {'Name': ['Aman', 'Riya', 'Karan'], 'Marks': [78, 88, 91], 'Age': [22, 21, 20]}
df = pd.DataFrame(data)
print(df[(df['Marks'] > 70) & (df['Age'] < 22)])
```

Q54. Extract rows where Marks > 90 or Age < 22.

Data

```
data = {'Name': ['Aman', 'Riya', 'Karan'], 'Marks': [78, 88, 91], 'Age': [22, 21, 20]}
```

Solution

```
import pandas as pd
data = {'Name': ['Aman', 'Riya', 'Karan'], 'Marks': [78, 88, 91], 'Age': [22, 21, 20]}
df = pd.DataFrame(data)
print(df[(df['Marks'] > 90) | (df['Age'] < 22)])
```

Q55. Extract rows where Marks are not greater than 80 using NOT operator.

Data

```
data = {'Name': ['Aman', 'Riya', 'Karan'], 'Marks': [78, 88, 91]}
```

Solution

```
import pandas as pd
data = {'Name': ['Aman', 'Riya', 'Karan'], 'Marks': [78, 88, 91]}
df = pd.DataFrame(data)
print(df[~(df['Marks'] > 80)])
```

Section G - Charts (Q56-Q60)

Q56. Create a line plot from the given DataFrame.

Data

```
data = {'Month': ['Jan', 'Feb', 'Mar', 'Apr'], 'Sales': [100, 150, 130, 170]}
```

Solution

```
import pandas as pd
data = {'Month': ['Jan', 'Feb', 'Mar', 'Apr'], 'Sales': [100, 150, 130, 170]}
df = pd.DataFrame(data)
df.plot(x='Month', y='Sales', kind='line')
```


Q57. Create a bar chart from the given DataFrame.

Data

```
data = {'Student': ['A', 'B', 'C'], 'Marks': [78, 88, 91]}
```

Solution

```
import pandas as pd
data = {'Student': ['A', 'B', 'C'], 'Marks': [78, 88, 91]}
df = pd.DataFrame(data)
df.plot(x='Student', y='Marks', kind='bar')
```

Q58. Create a histogram from the given DataFrame.

Data

```
data = {'Marks': [45, 56, 67, 78, 89, 90, 76, 65]}
```

Solution

```
import pandas as pd
data = {'Marks': [45, 56, 67, 78, 89, 90, 76, 65]}
df = pd.DataFrame(data)
df.plot(kind='hist')
```

Q59. Create a box plot from the given DataFrame.

Data

```
data = {'Marks': [45, 56, 67, 78, 89, 90, 76, 65]}
```

Solution

```
import pandas as pd
data = {'Marks': [45, 56, 67, 78, 89, 90, 76, 65]}
df = pd.DataFrame(data)
df.plot(kind='box')
```

Q60. Create an area plot from the given DataFrame.

Data

```
data = {  
    'Month': ['Jan', 'Feb', 'Mar', 'Apr'],  
    'ProductA': [10, 20, 15, 25],  
    'ProductB': [5, 15, 10, 20]  
}
```

Solution

```
import pandas as pd  
data = {  
    'Month': ['Jan', 'Feb', 'Mar', 'Apr'],  
    'ProductA': [10, 20, 15, 25],  
    'ProductB': [5, 15, 10, 20]  
}  
df = pd.DataFrame(data)  
df.plot(x='Month', kind='area')
```